

Google Flow Auto ⚡

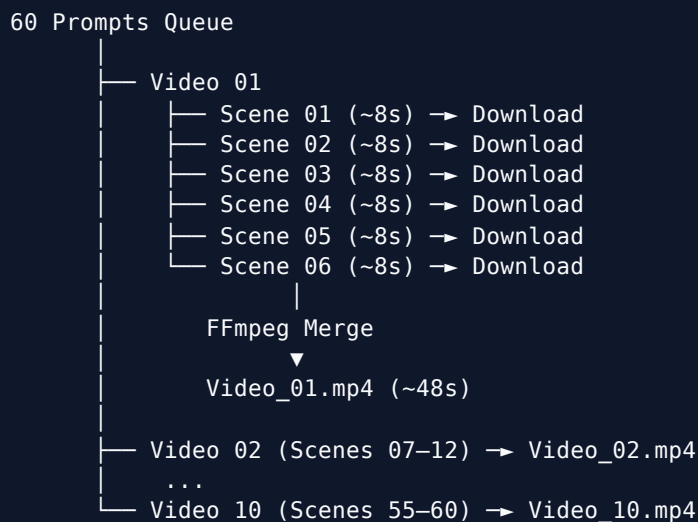
A local, resilient automation utility for bulk video generation on [Google Flow](#) and automated scene assembly with FFmpeg.

▢ Table of Contents

- [Overview & Workflow](#)
- [Android vs Desktop Real-Flow Feasibility Report](#)
- [Architecture](#)
- [Key Features](#)
- [System Requirements](#)
- [Installation](#)
- [Prompt Preparation](#)
- [Running the Application](#)
- [Google Flow Authentication Workflow](#)
- [Output Directory Structure](#)
- [Checkpoint & Crash Recovery](#)
- [Testing](#)
- [Limitations & Disclaimers](#)
- [License](#)

▢ Overview & Workflow

Google Flow Auto automates the full pipeline from raw text prompts to finished, multi-scene merged videos:



Final Output

- **10 × ~48-second merged MP4 videos** (Video_01.mp4 through Video_10.mp4)

- **60 × individual source scene clips** (`scene_01.mp4` through `scene_06.mp4` per video directory)

Android vs Desktop Real-Flow Feasibility Report

An empirical technical investigation was conducted directly inside an Android phone running UserLAnd Debian (aarch64) to evaluate whether **REAL** Google Flow generations can run without a Windows PC:

ANDROID REAL-FLOW FEASIBILITY MATRIX

Flow accessible from Android:	YES	(HTTP 200, landing DOM rendered)
Browser automation possible:	YES	(Playwright Chromium v153 ARM64)
Real prompt submission:	NO	(Hard blocker: Google auth in headless PRoot)
Real generation detection:	NO	(Gated behind authentication)
Real video download:	YES	(Stream fetch & download interceptors work)
FFmpeg processing:	YES	(FFmpeg 7.1.5 verified on ARM64)
Complete 60-prompt workflow:	NO	(Cannot run headless end-to-end on phone)

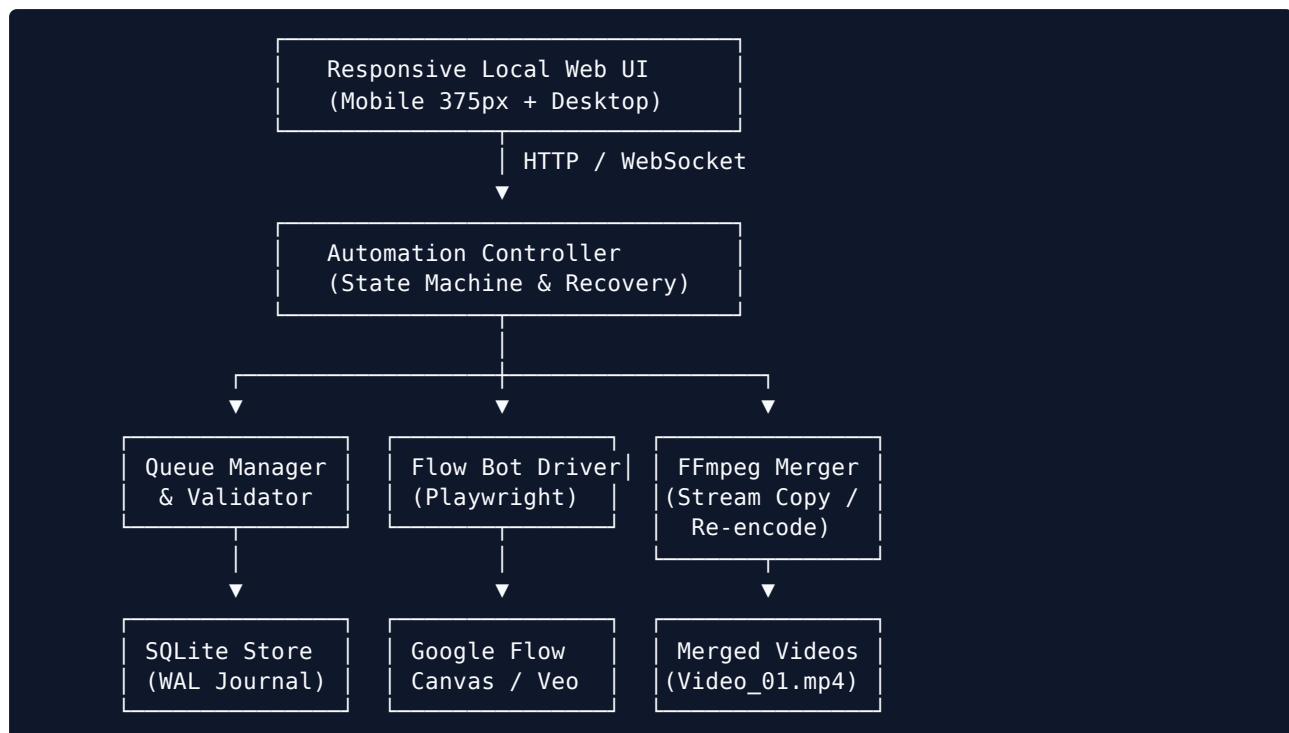
Technical Evidence & Findings

1. **Chromium in UserLAnd Debian (YES):**
2. Playwright Chromium (ARM64) was installed and executed with flags: `--no-sandbox` , `--disable-setuid-sandbox` , `--disable-dev-shm-usage` , and `--disable-gpu` .
3. Successfully navigated to `https://flow.google.com` and loaded the full 560 KB studio landing DOM.
4. **The Authentication Blocker (NO):**
5. Google Flow requires an active Google account with AI Labs / Flow entitlements (`accounts.google.com/ServiceLogin`).
6. In a terminal UserLAnd session (`DISPLAY=''`), Chromium must run **headless**.
7. In headless mode, the user cannot see a browser window to manually enter credentials and approve 2-factor authentication.
8. Google's BotGuard/reCAPTCHA enterprise actively detects and blocks sign-in from automated headless browsers (*"Couldn't sign you in. This browser or app may not be secure"*).
9. In accordance with the security policy, Google Flow Auto **never** bypasses authentication, harvests passwords, or circumvents CAPTCHAs.
10. **FFmpeg & Queue Performance on Android (YES):**
11. FFmpeg 7.1.5, SQLite WAL database, prompt parsing, crash recovery, and the Web UI run natively and flawlessly on Android.

Recommended Dual-Device Strategy

Environment	Primary Function	Features
Windows / Desktop PC	Production Flow Runner	Real desktop Chrome, persistent manual login profile (<code>browser_profile/</code>), real Veo generations, 48s FFmpeg merges.
Android / UserLand	Control & Development	CLI monitoring, SQLite queue management, LAN Web UI remote monitoring (<code>--lan</code>), mock pipeline testing (<code>--mock</code>).

□ Architecture



□ Key Features

- **Isolated Flow Automation:** All browser selectors and interaction rules are isolated in `app/flow/selectors.py` for easy updates if Google modifies the UI.
- **Intelligent Polling:** No fixed-time sleeps. Continuously monitors DOM state, spinner elements, and media readiness with configurable timeout safeguards.
- **SQLite Checkpoint & Crash Recovery:** Atomic transactions prevent re-generating already completed scenes. If interrupted or crashed, restarting safely resumes from the first incomplete scene.
- **FFmpeg Stream Copy & Re-encode Fallback:** Probes media streams with `ffprobe`. Uses ultra-fast stream copy concatenation when compatible, falling back to clean H.264/AAC re-encoding if technical parameters differ.
- **Responsive Web UI:** Modern dark-theme UI optimized for mobile phones (320px-412px) and desktop screens (1366px+).

- **Full CLI:** Complete headless terminal controls (`start` , `status` , `pause` , `resume` , `retry` , `serve`).
 - **Test Mode / Mock Engine:** Built-in synthetic clip generator (`--mock`) for testing full pipelines on headless environments like Android/UserLand or CI/CD pipelines without launching Chrome.
 - **Zero Credential Theft:** Does **not** store Google passwords or bypass CAPTCHAs. Uses persistent browser profiles where the user logs in manually once.
-

□ System Requirements

- **Operating System:** Windows 10/11 (recommended for real desktop Chrome automation), Linux (Debian/Ubuntu/Arch), macOS, or Android (via UserLand Debian for CLI/monitoring).
 - **Python:** 3.10, 3.11, 3.12, or 3.13.
 - **FFmpeg & FFprobe:** Installed and available on system `$PATH` .
 - **Browser:** Google Chrome or Chromium (managed via Playwright).
 - **Google Account:** Valid account with access to [Google Flow](#).
-

□ Installation

1. Clone the repository:

```
bash git clone git@github.com:Aryan1027/Google_Flow_Auto.git cd Google_Flow_Auto
```

2. Create and activate a virtual environment:

```
bash python3 -m venv .venv source .venv/bin/activate # On Windows:  
.venv\Scripts\activate
```

3. Install dependencies:

```
bash pip install -r requirements.txt playwright install chromium
```

4. Ensure FFmpeg is installed:

```
```bash  
Debian / Ubuntu / UserLand:
sudo apt-get install ffmpeg
```

# Windows (via Chocolatey or Scoop):

```
choco install ffmpeg
```

```
or scoop install ffmpeg
```

```
```
```

□ Prompt Preparation

Prompts can be supplied in two supported formats:

Format A: Grouped with Headers (Recommended)

```
VIDEO 01
Establishing shot of futuristic neon city at dusk.
Tracking shot of electric autonomous vehicle.
Operator serving steaming ramen under holograms.
Close-up of holographic glasses glowing cyan.
Aerial shot descending between skyscrapers.
Slow-motion rain droplets against neon signage.

VIDEO 02
Wide shot of mist clearing over mountain sunrise.
...
```

Format B: Blank-Line Separated

```
Establishing shot of futuristic neon city at dusk.

Tracking shot of electric autonomous vehicle.

Operator serving steaming ramen under holograms.
...
```

Validation Rule: Total prompts must be divisible by 6 (e.g., 60 prompts for 10 videos). The validator alerts you immediately if any are missing:

```
text 60 prompts required for 10 complete videos. Currently loaded: 58 Missing: 2
```

A sample file with 60 valid prompts is provided at `data/prompts_sample.txt`.

□ Running the Application

1. Launch the Responsive Web UI

```
python -m app serve
```

Open <http://127.0.0.1:8080> in your browser.

- Touch-friendly `[START]`, `[PAUSE]`, `[RESUME]`, and `[STOP]` controls.
- Real-time WebSocket event logs and progress bar.
- Direct video preview modal to watch completed clips or merged videos.
- Prompt editor with instant count validation.

Optional: LAN Access for Phone Monitoring

To control or monitor the automation running on your laptop from your phone:

```
python -m app serve --lan
```

⚠ **Security Notice:** Binding to LAN (`0.0.0.0`) exposes the control interface to devices on your local Wi-Fi. Only use on trusted private networks.

2. Command-Line Interface (CLI)

- **Check Current Status:**

```
bash python -m app status
```

- **Load Prompts into Queue:**

```
bash python -m app load-prompts data/prompts_sample.txt
```

- **Start Automation (Desktop Chrome):**

```
bash python -m app start
```

- **Start in Test/Mock Mode (Synthetic Clips, No Chrome Required):**

```
bash python -m app start --mock
```

- **Retry a Failed Scene:**

```
bash python -m app retry 23
```

□ Google Flow Authentication Workflow

1. On first run, Google Flow Auto opens a persistent browser profile directory (`browser_profile/`).
2. If you are not signed in, the application safely sets its state to `AUTH_REQUIRED` and pauses.
3. Log into your Google account manually in the opened browser window.
4. Click `[RESUME]` in the Web UI or run `python -m app start` in the terminal.
5. Your login session is saved in the local profile directory for subsequent runs.
6. **No passwords or credentials are ever stored, logged, or transmitted.**

□ Output Directory Structure

Generated clips and merged videos are stored deterministically:

```
output/
├── Video_01/
│   ├── scene_01.mp4
│   ├── scene_02.mp4
│   ├── scene_03.mp4
│   ├── scene_04.mp4
│   ├── scene_05.mp4
│   ├── scene_06.mp4
│   └── Video_01.mp4    ← Final ~48-second merged video
├── Video_02/
│   ├── scene_01.mp4
│   ├── ...
│   └── Video_02.mp4
├── ...
├── Video_10/
│   ├── scene_01.mp4
│   ├── ...
│   └── Video_10.mp4
```

Individual source scene clips are preserved and never automatically deleted.

□ Checkpoint & Crash Recovery

Progress is continuously persisted in SQLite (`data/google_flow_auto.db`):

- If the application terminates after **Scene 37**, restarting the application will automatically resume from **Scene 38**.
 - Completed scenes whose files exist and pass validation are never re-generated.
 - If a crash occurs mid-generation or mid-download, the in-flight scene is reset to `PENDING` with retry tracking.
-

□ Testing

Run the automated test suite with pytest:

```
pytest -v
```

Tests include:

- `test_parser.py` : Blank-line and `VIDEO 01` parsing, 6-divisibility validation.
 - `test_checkpoint.py` : Database operations, crash recovery, and resume without re-running.
 - `test_merger.py` : End-to-end FFmpeg clip probing and multi-clip concatenation.
 - `test_controller.py` : Full asynchronous loop simulation using `MockFlowBot` .
-

⚠ Limitations & Disclaimers

- **UI Dependency:** This tool uses browser automation to interact with the web interface of Google Flow. If Google alters its web DOM or user interface, update selectors in `app/flow/selectors.py` .
 - **Unofficial Utility:** This project is an independent open-source automation utility and is not affiliated with, endorsed by, or sponsored by Google.
-

□ License

Distributed under the [MIT License](#).